

4. YOLO 미세조정(Fine Tuning) 개념 및 방법

1 객체인식 파인튜닝

1-1. 객체인식 파인튜닝 데이터셋

객체 검출(Object Detection)은 **이미지 + 박스 정보(Bounding Box Annotation)**가 꼭 필요합니다. 단순히 이미지와 클래스 라벨만 있는 분류(Classification)와 달리, 모델이 "어디에 무엇이 있는지"를 학습하려면 **좌표 정보**까지 있어야 합니다.

예를 들어, 사과 하나가 있는 이미지라면:

```
target = {  
  "boxes": torch.tensor([[34, 50, 200, 300]], dtype=torch.float32),  
  "labels": torch.tensor([1]) # 1=사과  
}
```

객체 인식 모델을 파인튜닝하기 위한 데이터셋 준비는 크게 **이미지 수집, 어노테이션(Annotation)**, 그리고 **데이터 분할**의 세 단계로 나뉩니다. 각 단계는 모델의 성능을 결정짓는 중요한 요소입니다.

1. 이미지 수집

모델이 인식해야 할 객체들이 포함된 이미지를 충분히 확보해야 합니다. 다양한 환경과 조건의 이미지를 포함하는 것이 중요합니다.

- **다양성:** 객체의 크기, 각도, 조명, 배경 등을 다양하게 구성해야 합니다.
- **개수:** 각 클래스(객체 종류)당 최소 수백 장의 이미지를 확보하는 것이 좋으며, 데이터가 많을수록 모델의 정확도가 높아집니다.
- **파일 형식:** JPG, PNG 등 일반적인 이미지 형식을 사용하며, 데이터의 일관성을 위해 이미지 크기를 동일하게 맞추는 것이 좋습니다.

2. 어노테이션 (Annotation)

수집한 이미지에 모델이 학습할 수 있도록 ****레이블(Label)****을 지정하고 ****바운딩 박스(Bounding Box)****를 그리는 과정입니다. . 이 작업은 전용 도구를 사용하면 효율적입니다.

- **바운딩 박스**: 이미지 내의 각 객체를 직사각형으로 둘러싸는 작업입니다. 박스의 좌표는 (xmin, ymin, xmax, ymax) 또는 (x_center, y_center, width, height) 형식으로 저장됩니다.
- **클래스 레이블**: 바운딩 박스에 해당하는 객체가 무엇인지(예: 'car', 'person')를 지정합니다.
- **어노테이션 도구**: [Labellmg](#), [Roboflow](#), [CVAT](#) 등 다양한 무료/유료 도구가 있습니다.
- **파일 형식**: 어노테이션 정보는 일반적으로 XML(VOC 형식), JSON(COCO 형식), 또는 텍스트 파일(YOLO 형식)로 저장됩니다.

3. 데이터 분할

전체 데이터셋을 **학습(Training)**, **검증(Validation)**, **테스트(Test)** 세 가지 용도로 나눕니다.

- **학습 데이터셋 (Training Set)**: 모델이 학습하는 데 사용되는 가장 큰 부분입니다. 보통 전체 데이터의 70~80%를 차지합니다.
- **검증 데이터셋 (Validation Set)**: 모델 학습 중에 성능을 평가하고 하이퍼파라미터를 조정하는 데 사용됩니다. 보통 10~15%를 차지합니다.
- **테스트 데이터셋 (Test Set)**: 최종 모델의 성능을 평가하는 데 사용됩니다. 이 데이터는 학습 과정에 전혀 사용되지 않아야 합니다. 보통 10~15%를 차지합니다.

주의사항: 각 데이터셋에 클래스별 이미지 분포가 고르게 포함되도록 분할해야 모델이 편향되지 않은 학습을 할 수 있습니다. [train_test_split](#) 과 같은 라이브러리 함수를 사용하면 편리합니다.

1-2. Annotation 제작 & 공개 데이터셋

학습용 데이터셋을 준비할 때 → **이미지 + 박스 좌표 + 클래스 ID** 조합이 필요합니다.

- 라벨링 방법
 - 수작업라벨링
 - [Labellmg](#) → Pascal VOC(XML) or YOLO(txt) 포맷
 - [Labelme](#) → JSON 포맷
 - CVAT → 대규모 라벨링 도구

- 자동 변환

- VOC, YOLO, COCO 등 포맷 간 변환 라이브러리 사용 가능
- PyTorch torchvision 모델은 **COCO 스타일**을 제일 잘 지원

- 공개 데이터셋

- **COCO (Common Objects in Context):** <https://cocodataset.org/#home>

- 80개 클래스 (사람, 자동차, 개, 고양이 등)

```
COCO_INSTANCE_CATEGORY_NAMES = [  
    '__background__', 'person', 'bicycle', 'car', 'motorcycle', 'airplane',  
    'bus',  
    'train', 'truck', 'boat', 'traffic light', 'fire hydrant', 'N/A', 'stop sign',  
    'parking meter', 'bench', 'bird', 'cat', 'dog', 'horse', 'sheep', 'cow',  
    'elephant', 'bear', 'zebra', 'giraffe', 'N/A', 'backpack', 'umbrella',  
    'N/A', 'N/A',  
    'handbag', 'tie', 'suitcase', 'frisbee', 'skis', 'snowboard', 'sports ba  
ll',  
    'kite', 'baseball bat', 'baseball glove', 'skateboard', 'surfboard', 'te  
nnis racket',  
    'bottle', 'N/A', 'wine glass', 'cup', 'fork', 'knife', 'spoon', 'bowl',  
    'banana', 'apple', 'sandwich', 'orange', 'broccoli', 'carrot', 'hot do  
g', 'pizza',  
    'donut', 'cake', 'chair', 'couch', 'potted plant', 'bed', 'N/A', 'dining t  
able',  
    'N/A', 'N/A', 'toilet', 'N/A', 'tv', 'laptop', 'mouse', 'remote', 'keyboar  
d', 'cell phone',  
    'microwave', 'oven', 'toaster', 'sink', 'refrigerator', 'N/A', 'book',  
    'clock', 'vase', 'scissors', 'teddy bear', 'hair drier', 'toothbrush'  
]
```

- 박스, 세그멘테이션, Keypoint 주석까지 제공
- PyTorch `torchvision.datasets.CocoDetection` 으로 바로 로드 가능

```
{  
    "images": [  

```

```

    {"id": 0, "file_name": "image1.jpg", "width": 640, "height": 480},
    ...
  ],
  "annotations": [
    {
      "id": 0,
      "image_id": 0,
      "category_id": 1,
      "bbox": [100, 50, 200, 300],
      "area": 60000,
      "iscrowd": 0
    },
    ...
  ],
  "categories": [
    {"id": 1, "name": "object_name_1"},
    ...
  ]
}

```

- **Pascal VOC:**

<https://storage.googleapis.com/openimages/web/index.html>

- 20개 클래스
- 라벨은 XML (VOC 포맷)
- 상대적으로 가볍고 연습용으로 적합

- **Open Images Dataset (Google):**

<https://storage.googleapis.com/openimages/web/index.html>

- 600개 이상의 클래스
- 매우 대규모 (COCO보다 훨씬 큼)
- 필요 클래스만 필터링 가능

- **Kaggle 데이터셋:** [https://www.kaggle.com/datasets?](https://www.kaggle.com/datasets?search=object+detection)

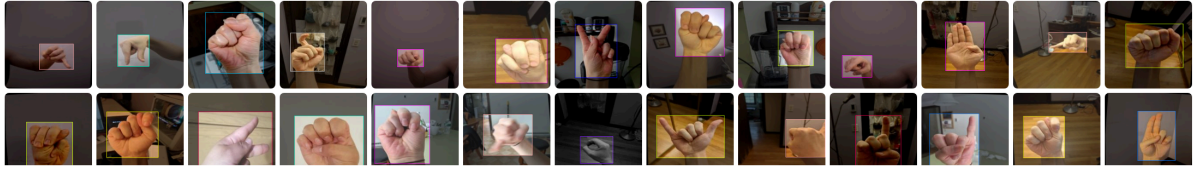
[search=object+detection](https://www.kaggle.com/datasets?search=object+detection)

- 작은 규모의 라벨링 데이터셋 다양 (마스크 착용, 교통 표지판, 동물 등)

1-3. 로보플로우 데이터셋 다운로드

<https://public.roboflow.com/object-detection/american-sign-language-letters>

Preview



1

upload

Images and Annotation Format: COCO

Download Options

Download zip to computer

Show download code

Cancel

Continue

2

download

Use this code to download and unzip your dataset via the command line on any "nix machine":

curl -L "https://public.roboflow.com/ds/v8l89y9pM8?key=cj5djs0qmf8d" -o roboflow.zip; unzip roboflow.zip; rm roboflow.zip

Done

Choose a Model

3

object

test

train

valid

README.roboflow.txt

1

curl -L "https://public.roboflow.com/ds/v8l89y9pM8?key=cj5djs0qmf8d" -o roboflow.zip

2

mkdir -p object

3

unzip roboflow.zip -d object

4

rm roboflow.zip

extracting: object/valid/821.jpg.rf.5ac296aea177a402b5a511f0d51f6154.jpg

extracting: object/valid/f19.jpg.rf.5c3511a00813d7c51e97711627291a52.jpg

extracting: object/valid/z11.jpg.rf.5b94b9ef0a821560013710759a627a78.jpg

extracting: object/valid/C1.jpg.rf.5711d988bbade2e73e810f6991b23.jpg

extracting: object/valid/P23.jpg.rf.5ba3399d298811d1fae185cb0d41a45c.jpg

4

dataset/

train/

images/ (여기에 .jpg, .png 같은 실제 이미지들)

_annotations.coco.json

valid/

images/

_annotations.coco.json

test/

images/

_annotations.coco.json

extracting: object/valid/f13.jpg.rf.ec0372bdc5e7e63cd9d03b551c9fb80b.jpg

extracting: object/valid/S22.jpg.rf.f4d4e014e423d20c9d2d1d566f5a33fa.jpg

extracting: object/valid/H18.jpg.rf.ff988df01210143b03fc11601dee2f5b.jpg

extracting: object/valid/D15.jpg.rf.f79cfd74547913cedee1cd9f8a19e450.jpg

extracting: object/test/_annotations.coco.json

extracting: object/train/_annotations.coco.json

extracting: object/valid/_annotations.coco.json

extracting: object/README.roboflow.txt

▼ train폴더의 _annotations.coco.json

```
{
  "info": {
    "year": "2020",
    "version": "1",
    "description": "Exported from roboflow.ai",
    "contributor": "",
    "url": "https://app.roboflow.ai/datasets/american-sign-language-letters/1",
    "date_created": "2020-10-20T16:55:24+00:00"
  },
  "licenses": [
    {
      "id": 1,
```

```
        "url": "",
        "name": "Unknown"
    }
],
"categories": [
    {
        "id": 0,
        "name": "Letters",
        "supercategory": "none"
    },
    {
        "id": 1,
        "name": "A",
        "supercategory": "Letters"
    },
    {
        "id": 2,
        "name": "B",
        "supercategory": "Letters"
    },
    {
        "id": 3,
        "name": "C",
        "supercategory": "Letters"
    },
    {
        "id": 4,
        "name": "D",
        "supercategory": "Letters"
    },
    {
        "id": 5,
        "name": "E",
        "supercategory": "Letters"
    },
    {
        "id": 6,
        "name": "F",
```

```
    "supercategory": "Letters"
  },
  {
    "id": 7,
    "name": "G",
    "supercategory": "Letters"
  },
  {
    "id": 8,
    "name": "H",
    "supercategory": "Letters"
  },
  {
    "id": 9,
    "name": "I",
    "supercategory": "Letters"
  },
  {
    "id": 10,
    "name": "J",
    "supercategory": "Letters"
  },
  {
    "id": 11,
    "name": "K",
    "supercategory": "Letters"
  },
  {
    "id": 12,
    "name": "L",
    "supercategory": "Letters"
  },
  {
    "id": 13,
    "name": "M",
    "supercategory": "Letters"
  },
  {
```

```
    "id": 14,  
    "name": "N",  
    "supercategory": "Letters"  
  },  
  {  
    "id": 15,  
    "name": "O",  
    "supercategory": "Letters"  
  },  
  {  
    "id": 16,  
    "name": "P",  
    "supercategory": "Letters"  
  },  
  {  
    "id": 17,  
    "name": "Q",  
    "supercategory": "Letters"  
  },  
  {  
    "id": 18,  
    "name": "R",  
    "supercategory": "Letters"  
  },  
  {  
    "id": 19,  
    "name": "S",  
    "supercategory": "Letters"  
  },  
  {  
    "id": 20,  
    "name": "T",  
    "supercategory": "Letters"  
  },  
  {  
    "id": 21,  
    "name": "U",  
    "supercategory": "Letters"
```



```

    },
    {
      "id": 22,
      "name": "V",
      "supercategory": "Letters"
    },
    {
      "id": 23,
      "name": "W",
      "supercategory": "Letters"
    },
    {
      "id": 24,
      "name": "X",
      "supercategory": "Letters"
    },
    {
      "id": 25,
      "name": "Y",
      "supercategory": "Letters"
    },
    {
      "id": 26,
      "name": "Z",
      "supercategory": "Letters"
    }
  ],
  "images": [
    {
      "id": 0,
      "license": 1,
      "file_name": "G7_jpg.rf.04faf434dd590a7bd02818b2b57a704f.jp
g",
      "height": 416,
      "width": 416,
      "date_captured": "2020-10-20T16:55:24+00:00"
    },
    {

```

```
    "id": 1,  
    "license": 1,  
    "file_name": "T24_jpg.rf.068e7b2424eac996c86bb6d9c38e083  
d.jpg",  
    "height": 416,  
    "width": 416,  
    "date_captured": "2020-10-20T16:55:24+00:00"  
  },
```

📌 README roboflow.txt

- 데이터셋 이름: **American Sign Language Letters - v1**
- 생성 날짜: 2020년 10월 20일 4:55 PM (GMT 기준)
- 총 이미지 수: **1,728장**
- 어노테이션 형식: **COCO** 형식으로, 각 글자의 위치가 바운딩 박스(Bounding Box)로 표시되어 있습니다.
- 적용된 전처리 (Pre-processing):
 1. 자동 방향 조정: EXIF 정보를 제거하여 이미지의 방향을 올바르게 맞춥니다.
 2. 크기 조정 (**416x416**): 모든 이미지를 **416x416** 픽셀 크기로 늘려(Stretch) 통일합니다. 이는 모델 입력에 맞추기 위한 일반적인 과정입니다.
- 적용된 증강 (Augmentation):
 - 원본 이미지 한 장당 **3가지 버전**의 증강된 이미지를 생성했습니다.
 - 수평 뒤집기 (Horizontal Flip): 50% 확률로 이미지를 좌우로 뒤집습니다.
 - 무작위 자르기 (Random Crop): 이미지의 0~20%를 무작위로 잘라냅니다.
 - 무작위 회전 (Random Rotation): -5도에서 +5도 사이로 이미지를 무작위로 회전시킵니다.
 - 무작위 기울기 (Random Shear): 수평 및 수직 방향으로 -5도에서 +5도 사이로 이미지를 기울입니다.
 - 무작위 밝기 조절 (Random Brightness): -25%에서 +25% 사이로 밝기를 조절합니다.
 - 무작위 가우시안 블러 (Random Gaussian blur): 0에서 1.25픽셀 사이의 가우시안 블러를 적용합니다.

→ 이 데이터셋은 ASL 글자를 인식하기 위해 **증강(augmentation)** 과정을 거쳐 **1,728장**의 이미지로 확장된 데이터입니다. 모델 학습 시, 이 데이터는 **416x416** 크기의 이미지를 COCO 형식의 바운딩 박스와 함께 사용하여 객체 탐지 모델을 훈련하는 데 적합합니다. 증강 기법을 통해 데이터 다양성을 높여 모델의 일반화 성능을 향상시키는 것이 목표입니다.

- 데이터셋이 크지 않으면 → **COCO pretrained + 마지막 헤드 교체**가 가장 안정적.
- 데이터셋이 매우 작으면 → **MobileNet 백본** (가벼움, 적은 데이터에도 학습 잘 됨).

- 데이터셋이 매우 크면 → **COCO pretrained 없이 from scratch**도 가능

- 소규모 데이터셋(1.7k장) → `fasterrcnn_mobilenet_v3_large_fpn` 권장.
- 사전학습(COCO) → head 교체 → backbone freeze 후 튜닝 → 전체 fine-tuning 단계 진행.
- 증강은 최소 (이미 Roboflow에서 충분히 적용됨).
- 목표: 빠른 수렴 + 작은 데이터에도 과적합 방지
- COCO모델로 학습한 사전학습 모델

```
from torchvision.models.detection import (
    fasterrcnn_resnet50_fpn,
    fasterrcnn_mobilenet_v3_large_fpn,
    retinanet_resnet50_fpn,
    maskrcnn_resnet50_fpn,
    keypoint_rcnn_resnet50_fpn,
)
```

2 아이디어: Mediapipe → 객체 인식 모델 미세조정

1. 문제 정의

- 회사에서 안전모 착용 여부를 자동으로 검출하는 시스템이 필요하다.
- 하지만, 기존에 라벨링된 "안전모 데이터셋"이 부족하다.
- 라벨링을 새로 하기엔 시간과 비용이 너무 크다.

2. Mediapipe 활용

- Mediapipe의 **Face Mesh**와 **Selfie Segmentation**을 활용해서,
 - 얼굴 위치 자동 검출 → 얼굴 영역 바운딩 박스 생성
 - 머리 부분(안전모 착용 위치)도 추출 가능

→ 즉, 사람이 직접 박스를 그리지 않고도 **라벨 자동 생성기**처럼 Mediapipe가 박스를 만들어 준다.

3. 데이터셋 준비

- 원본 CCTV 이미지들 10,000장 확보
 - Mediapipe 실행 → 각 이미지마다 얼굴/머리 좌표 추출
 - 좌표를 `[xmin, ymin, xmax, ymax]` 포맷으로 변환
 - 라벨은 두 가지 클래스:
 - 1 = 안전모 착용
 - 2 = 안전모 미착용
 - 이 정보를 PyTorch Detection Dataset 포맷으로 정리
-

4. 객체 인식 모델 미세조정

- 기본 모델: torchvision의 Faster R-CNN (ResNet50-FPN)
 - 학습 데이터: Mediapipe가 생성한 바운딩 박스 + 안전모 라벨
 - 전략:
 - 처음엔 Mediapipe 박스를 그대로 학습
 - 이후 소량의 "사람이 직접 라벨링한 이미지"로 **fine-tuning** 보정
-

5. 결과

- 모델이 "CCTV 영상에서 안전모 착용 여부"를 **실시간 탐지** 가능
 - Mediapipe 덕분에 **수천 장을 자동 라벨링** → 수작업 라벨링은 소량 보정만 필요
 - 최종적으로, 모델 성능이 사람이 직접 모든 데이터를 라벨링했을 때와 유사하게 도달
-

6. 확장 가능성

- 같은 방식으로 **작업자의 손 위치, 위험구역 진입 여부, 마스크 착용 여부** 등 다른 객체 인식 문제에도 적용 가능
- Mediapipe는 "라벨링 자동화"와 "전처리" 역할을 하고, Detection 모델은 "실제 인식"을 담당